

CPE 4669 Lab 3: Distributed Sorting with MPI

Team members: Ryan Salute, Winnie Trinh, Karina Lee, Gauransh Thapar

Algorithm Description

The program uses a distributed merge sort to sort N integers with MPI. Rank 0 generates the input using a fixed random seed so that every run uses reproducible data. The integers are divided as evenly as possible among all MPI ranks. When N is not divisible by the rank count, the first ranks receive one additional integer. Rank 0 uses counts and displacement arrays with `MPI_Scatterv` to distribute these uneven chunks.

Each rank sorts its local chunk with C's `qsort` function. The sorted chunks are then combined using a binary merge tree. During the first round, ranks 1, 3, 5, and so on send their chunks to ranks 0, 2, 4, and so on. Each receiver performs a linear merge of its local sorted chunk and the received sorted chunk. The distance between communication partners doubles after every round. This continues until rank 0 owns all N integers in globally sorted order.

The merge tree supports non-power-of-two rank counts. If a receiver does not have a partner in a particular round, it keeps its current data and continues to the next round. A sending rank first transmits its element count so that the receiver can allocate enough memory, and then sends its sorted data.

The timed region begins after input generation and an `MPI_Barrier`. It includes input distribution, local sorting, and all merge-tree communication and merging. Input generation, correctness verification, and printing are excluded. Each rank measures its elapsed time, and `MPI_Reduce` with `MPI_MAX` reports the maximum time across all ranks.

Experimental Environment

Partition: compute

Nodes used: 1

MPI tasks per node: up to 128

Input size: $N = 10,000,000$

Trials per rank count: 3

Compiler flags: `-O3 -std=c11 -Wall -Wextra -Wpedantic`

Correctness Evidence

Rank 0 verifies the final result in three ways. First, the number of integers after the merge must equal N. Second, every adjacent pair must be in nondecreasing order. Third, the additive checksum of the final array must match the checksum of the original input. Together, these checks detect an incorrectly ordered result as well as values that were lost, duplicated, or changed during communication.

The program was tested with one rank, a non-power-of-two rank count, uneven data distribution, and an empty input. The required test results were:

```
mpirun -np 1 ./mpi_sort 0
SUMMARY N=0 ranks=1 seconds=0.000002916 status=PASS
mpirun -np 3 ./mpi_sort 17
SUMMARY N=17 ranks=3 seconds=0.000010049 status=PASS
mpirun -np 5 ./mpi_sort 100003
SUMMARY N=100003 ranks=5 seconds=0.003002648 status=PASS
```

Results

Each rank configuration was run 3 times with the same input size, $N = 10000000$. The median execution time from those trials was used as T_p . Speedup and parallel efficiency were calculated using:

$$S(p) = T_1 / T_p$$

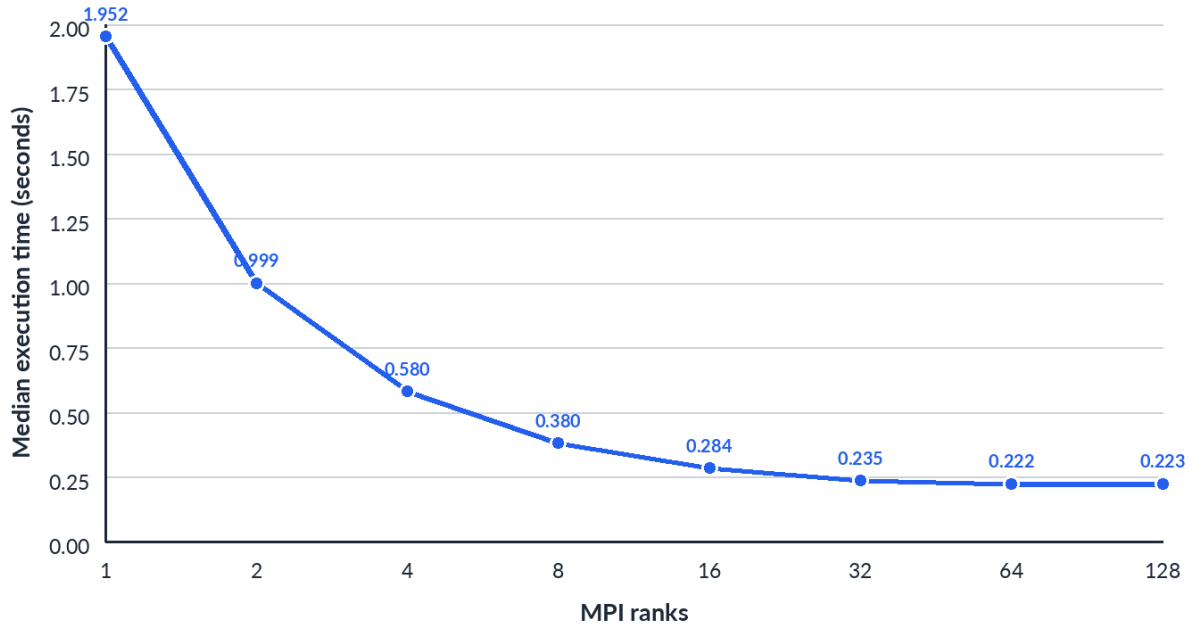
$$E(p) = S(p) / p$$

Ranks	Median time (s)	Speedup	Efficiency	Correct?
---: ---: ---: ---: ---:				
1	1.952094725	1.000	1.000	PASS
2	0.998622386	1.955	0.977	PASS
4	0.580407337	3.363	0.841	PASS
8	0.379671735	5.142	0.643	PASS
16	0.283832983	6.878	0.430	PASS
32	0.234579634	8.322	0.260	PASS
64	0.222427800	8.776	0.137	PASS
128	0.222711513	8.765	0.068	PASS

Strong-Scaling Plot

MPI Merge Sort Strong Scaling

N = 10,000,000; median of 3 trials



Scalability Discussion

Performance improved as the rank count increased from 1 to 64. The best result was 0.222428 seconds with 64 ranks, giving a speedup of 8.776. Increasing to 128 ranks did not improve performance, and efficiency fell from 13.7% at 64 ranks to 6.8% at 128 ranks. This indicates that communication, merging, and memory overhead outweighed the benefit of dividing the local sorting work among more ranks.